

Professores:

Celso Giusti

Daniel Manoel Filho

Felipe Augusto Stevanim Gregate

Marlon Palata Fanger Rodrigues

Formulários e Listas



Capacidades Técnicas Abordadas

3. Interpretar os requisitos do sistema, tendo em vista a elaboração dos componentes em ambiente mobile
4. Definir os elementos de entrada, processamento e saída para a codificação das funcionalidades mobile
5. Projetar interfaces para dispositivos móveis
6. Implementar o código respeitando as características da linguagem na plataforma mobile

O que vamos construir hoje?

Hoje vamos revisar nosso conhecimento no TextInput e conhecer novos componentes de lista.

Para isso vamos criar um app de Lista de Compras.

Por que isso importa? Todo app real coleta dados do usuário e exibe listas — Instagram, iFood, WhatsApp, Trello. Hoje você aprende os dois blocos fundamentais para isso.

O que vamos utilizar?

- **TextInput** controlado com **useState**;
- Múltiplos campos no mesmo formulário;
- Validação básica;
- **FlatList** para renderizar a lista.

Relembrando



Revisão Rápida – TextInput

Revisão sobre algumas propriedades do componente **TextInput**:

- **value** – o TextInput exibe o que está no estado;
- **onChangeText** – atualiza o estado a cada letra digitada;
- Sem o value, o campo e o estado ficam fora de sincronia.

```
const [texto, setTexto] = useState('');  
  
<TextInput  
  value={texto}  
  onChangeText={(novoValor) => setTexto(novoValor)}  
  placeholder="Digite algo..."  
>
```

Dica: também podemos fazer desta forma: **onChangeText={setTexto}**

Multiplos Campos

Para um formulário com diversos campos, precisamos de um **useState** para cada campo:

- **Nome:** setName
- **Sobrenome:** setSobrenome
- **CPF:** setCpf
- **Idade:** setIdade
- **Profissão:** setProfissao



KeyboardType

Outra propriedade importante para qualidade de vida do usuário é o KeyboardType, onde podemos definir qual o tipo padrão de teclado que vai ser utilizado no campo.

- **default:** Texto padrão;
- **numeric:** Apenas com números;
- **email-adress:** Texto com @ para e-mails
- **phone-pad:** Para telefone

Validação Básica

Antes de processar o formulário, é necessário verificar se os campos estão preenchidos, para fazer isso podemos utilizar:

- `trim()` - Remove os espaços em branco do início e do fim
- `Alert` - exibe um alerta informando o usuário

```
function handleAdicionar() {  
  if (tarefa.trim() === '') {  
    Alert.alert('Atenção', 'O nome da tarefa não pode ficar vazio!');  
    return;  
  }  
  
  // continua o processamento...  
  setTarefa('');  
}
```


Exemplo Formulário – Código

```
export default function Form() {  
  const [tarefa, setTarefa] = useState("");  
  const [prioridade, setPrioridade] = useState("");  
  
  function handleAdicionar() {  
    if (tarefa.trim() === "") {  
      Alert.alert("Atenção", "O nome da tarefa não pode ser vázia");  
      return;  
    }  
    setTarefa("");  
    setPrioridade("");  
  }  
}
```

Exemplo Formulário – Código

```
<TextInput
  value={tarefa}
  onChangeText={setTarefa}
  placeholder="Nome da tarefa"
/>
<TextInput
  value={prioridade}
  onChangeText={setPrioridade}
  placeholder="Prioridade (alta, média, baixa)"
/>

<TouchableOpacity onPress={handleAdicionar}>
  <Text>Adicionar</Text>
</TouchableOpacity>
```

FlatList



O que é Flatlist?

Flatlist é um **componente** oficial do React Native para renderizar listas longas com eficiência.

Nós estudamos e utilizamos o **.map()** no início mas o ideal é não utilizá-lo para isso.

O **map()** renderiza todos os itens de uma vez na memória. Com 1000 itens, 1000 componentes são criados ao mesmo tempo.

A **FlatList** é inteligente: ela só renderiza os itens visíveis na tela + alguns de reserva. Os outros são descartados da memória e recriados quando o usuário rolar.

Esqueleto da FlatList

Principais propriedades:

- **data**: array de itens que serão exibidos;
- **keyExtractor**: função que retorna uma chave única por item;
- **renderItem**: função que retorna JSX de cada item.

```
import { FlatList } from 'react-native';

<FlatList
  data={tarefas}
  keyExtractor={(item) => item.id}
  renderItem={({ item }) => (
    <Text>{item.nome}</Text>
  )}
/>
```


keyExtractor – Importante

O React Native precisa identificar cada item da lista de forma única para saber o que mudou, o que foi adicionado e o que foi removido sem ter que recriar tudo do zero.

O problema sem a key única:

- Lista tem: [A, B, C];
- Usuário remove B
- **Sem key:** React não sabe qual foi removido e pode re-renderizar tudo;
- **Com key:** React sabe exatamente que B saiu, só atualiza o necessário.

renderItem – Importante

O **renderItem** recebe um objeto com a prop **item** (o elemento atual do array):

Aqui você vai escrever a estrutura que deverá ser exibida em cada **ITEM** da lista.

```
renderItem={({ item }) => (  
  <View style={styles.card}>  
    <Text style={styles.nomeTarefa}>{item.nome}</Text>  
    <Text style={styles.prioridade}>{item.prioridade}</Text>  
  </View>  
)}
```


renderItem - Componente

O ideal nessa situação é componentizar o item que será renderizado.

```
<FlatList
  data={alunos}
  keyExtractor={(item) => item.id}
  renderItem={({ item }) => (
    <CardItem nome={item.nome} nota={item.nota}/>
  )}
/>
```

Atividade

Exercício 1



Desafio, irei mostrar a lista de tarefas já feita e comentar o que precisa ser feito, vocês irão anotar tudo que eu falar e executar a atividade.

Aquecimento Crie um componente **ListaDeCompras** com um **TextInput** e um **botão** "Adicionar".

Cada item digitado deve aparecer numa **FlatList** simples com apenas o nome do produto. Use **Date.now().toString()** como id.

Exercício 2

Validação no componente do exercício 1, adicione validação:

- Não permitir adicionar item com campo vazio
- Exibir Alert com a mensagem: "Digite o nome do produto antes de adicionar."
- Limpar o campo após adicionar com sucesso

Exercício 3

Formulário completo Adicione um segundo campo ao formulário:

quantidade(keyboardType="numeric").

Cada item da lista deve exibir: "2x Arroz" (quantidade + nome).

Valide que quantidade não está vazia também.

Atividade

Exercício 4



Card com estilo Estilize o renderItem da lista:

- Cada item em um card branco com borderRadius e sombra (elevation: 2)
- Nome em negrito
- Use StyleSheet.create

Exercício 5

Lista vazia Adicione um **ListEmptyComponent** personalizado à FlatList:

- Exibir um Text centralizado com a mensagem: "Sua lista está vazia. Adicione um produto!"
- Estilizar com cor cinza e marginTop

Referências

FACEBOOK INC. FlatList — React Native Documentation. Disponível em: <https://reactnative.dev/docs/flatlist>. Acesso em: abr. 2026.

FACEBOOK INC. TextInput — React Native Documentation. Disponível em: <https://reactnative.dev/docs/textinput>. Acesso em: abr. 2026.

FACEBOOK INC. Alert — React Native Documentation. Disponível em: <https://reactnative.dev/docs/alert>. Acesso em: abr. 2026.

EXPO. Expo Documentation. Disponível em: <https://docs.expo.dev>. Acesso em: abr. 2026.



ESCOLA SENAI ÍTALO BOLOGNA

Av. Goiás, 139

Telefone

(11) 2396-1999

Instagram

@senai.itu

Facebook

/senaisp.itu

Site

<https://sp.senai.br/unidade/itu/>